Intro to ML

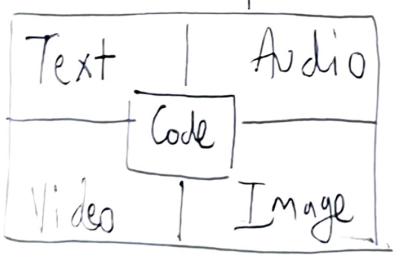
Powers some of the most important & Op technologies that we use. A way to
- → Solve problems
- → Answer complex question
- → Create new content

Basically
ML → process of training a piece of software, called model, to make useful predictions or generate content

Main task/ability is to learn about the patterns.
Connections & correlations

Types
1) Supervised → Regression → Classification
2) Unsupervised → Clustering, Association, Dimensionality Reduction
3) Reinforcement → Makes predictions based on getting rewards / penalties based on actions performed within an environment. Eg: AlphaGo
4) Gen AI

Creates Unique Content based on users' input



To produce unique & creative outputs, models are initially trained using unsupervised learning then by supervised/reinforcement for perfecting
i.e. Semi-supervised used a lot

Main Concepts of Supervised Learning

1) Data → Text, Audio, Image, table → Features + Label
2) Model → Complex set of no.s defining relationship b/w i/p & o/p
3) Training → Updating Model Parameters → From given dataset, its model's role to work out the best soln for predicting labels. This is done by training done by
4) Evaluating
5) Inference

Features characterized by its size (diversity i.e. no. of examples (range of examples)

Large size & diversity better

Comparing predicted & Actual Values

100 years of rainfall data but most is for June, we can't use it for January

Evaluating → How well model has learned
Inference → Once satisfied with evaluation results, we use the model for predictions called as inferences

Linear Regression

↓

Statistical Technique used to find relation b/w variables.

↓

Here b/w features & labels

$y = mx + b$ → in algebraic terms

$y' = b + w_1 x_1$ → in ML terms

- y' → Prediction
- b → Bias
- w_1 → weight
- x_1 → Feature Value

For Multiple features,

$$y' = b + w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + w_5 x_5 + \dots$$

Loss → tells How wrong predictions are

↳ Focuses on distance & not direction

Types

1) L_1 Loss $\sum |\text{actual} - \text{predicted}|$

2) Mean Absolute Error (MAE) $\frac{1}{N} L_1$ Loss (i.e. Avg)

3) L_2 Loss $\sum (\text{actual} - \text{predicted})^2$

4) Mean Squared Error (MSE) $\frac{1}{N} L_2$ Loss (i.e. Avg)

Makes bigger values stand out & smaller values to diminish

1.1 → 1.21

0.9 → 0.81

MAE, MSE → Preffered for multiple examples as they are avg.s

ML (20)

Local Minima

↓

No (only global)

In (30) DL → Local & Global

issues solved by optimizers like RMSprop, Adam

MAE (or) MSE → Depends on dataset

↓

Choose based on how you want model to treat outliers

MSE → moves model towards outliers, farther from most points

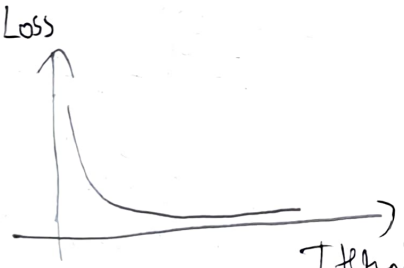
MAE → does not, closer to most points

Gradient Descent → Mathematical Technique that finds best weights & bias

Steps:

Initialize W & B to random no.s near zero, then

1) Calculate loss
2) Determine dir^n of movement] by Calculus
3) Move in that dir^n & see loss]
4) Return to Step 1 till finished



Closer to 1 better, 0 → mean, -ve → bad

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

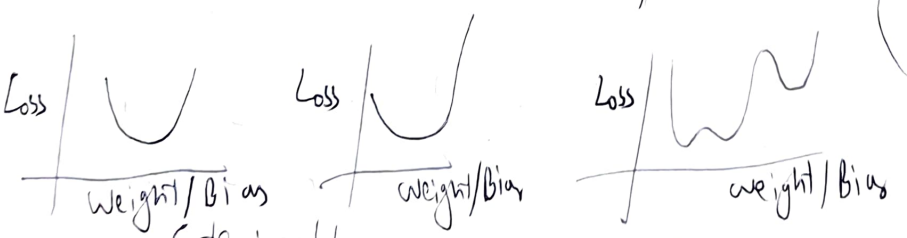
R^2 Adjusted $= 1 - \frac{(1-R^2)(N-1)}{N-P-1}$

better than R^2 since R^2 will better even for useless features

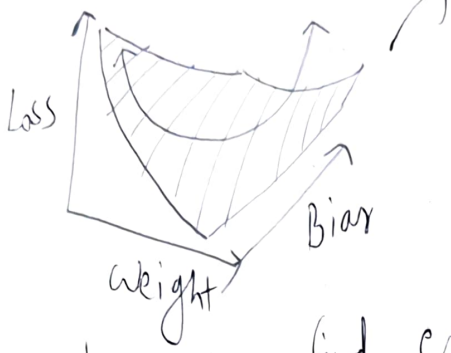
N → No. of datapoints, P → No. of predictors

Convergence

Loss f^ of Linear Model produce Convex Surface ('U' Shaped)



In 3D,



Like a paper & we try to find the bottom point of this paper

Model never find exact lowest point but a point very close to it

Hyperparameters → Variables that we control & not a part of model itself

- Learning Rate → Influences how quickly model converges. Very High, it bounces like crazy
- Batch Size → No. of Examples, model processes before updating weights & biases
- Epochs → Model has processed each example in training set once
  Eg: 1000 examples with mini batch of 100 examples → 10 iterations to complete 1 epoch

Stochastic Gradient Descent &

- Single Example i.e. batch Size of 1 per iteration
- Works but very "noisy"

for, FBGD → 1000 examples, 20 epochs
20 times Ws & Bs change

SGD → 1000 examples, 20 epochs
20,000 times Ws & Bs change

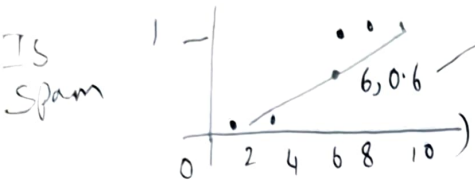
MBGD → 1000 examples, 100 per batch
20 epoch
200 times Ws & Bs change

Mini Batch Gradient Stochastic Descent

- b/w Full Batch & Stochastic
- batch size > 1 & batch size < N
- Chooses Examples & averages their gradient
- Small batch size behave like SGD & Larger batch size like FBGD

Logistic Regression

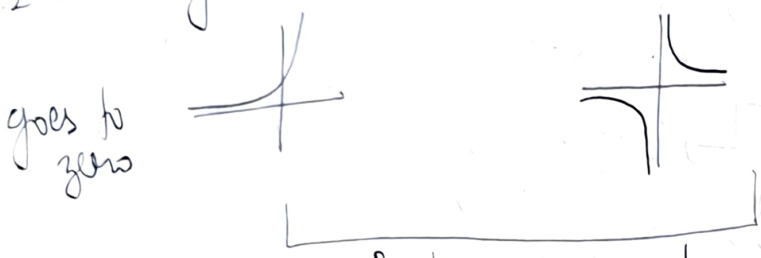
Say it be choose b/w 2 like 0, 1



→ the likelihood of spam
Also line goes further than 1 &
Lower than 0

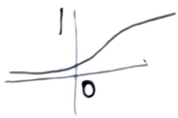
But predicting probability directly would be more better for computation & understanding data

2 f^n's $g(x) = e^x$ $g(x) = 1/x$ (hyperbola)



Both converge to 0 values but not in range of 0 & 1

So Combine



Sigmoid → One of family of Logistic Curves

$$f(x) = \frac{1}{1+e^{-x}}$$

→ A Linear Line

Say, $z = b + w_1 x_1 + \dots w_n x_n$

then $y' = \frac{1}{1+e^{-z}}$ $\Rightarrow z = \ln\left(\frac{y}{1-y}\right)$

odds ↓ (log-odds)

Bending the Linear Line/graph

z

y

Sending Graph into probabilities

Here training ⟶ Similar to Linear Regression

except

- ⟶ Loss ⟶ Log Loss instead of Squared/absolute loss
- ⟶ Apply of Regularization

$$\text{Log Loss} = \sum -y \log(y') - (1-y) \log(1-y')$$

Since Rate of Change of o/p is not Same throughout

y ⟶ Actual

y' ⟶ Predicted

Regularization ⟶ mechanism for reducing model Complexity during training

Extremely imp^ for Logistic Regression ⟶ As Asymptotic nature of the f^n will make loss to 0 for Large features ⟶ Basically leads to overfitting

Types:

- L_2
- Early Stopping

Classification → Which of the set of classes, the Example belong to

Logistic Regression → % of being likely

% → to classify → we use classification threshold

Eg: > 50% ↑

↓

depends on distribution. if Spam is like 0.01% of all, then threshold should be like 0.9.

If equal to class^n threshold, depends on tools used like keras considers it as below 0.5 i.e. negative class

0.5 → May not be good idea

Just Probality score for assesing the model is not accurately telling the whole truth That is why in Binary Class^n we use Confusion Matrix

Predicted \ Actual	+ve	-ve
+ve	TP	FP
-ve	FN	TN

TP → Spam identified as spam

TN → not spam identified as not spam

FP → Not Spam as Spam

FN → Spam as not Spam

For better threshold look at the FP & FN values, they should be low & relatively same

TP, TN, FP, FN → used to calculate different metrics

↓

each metric's usefullness depends on model, task, etc.

Accuracy = Correct / Total

= TP + TN / TP + TN + FP + FN → fraction of all emails correctly classified as spam/not

↓

for high level model Assement → Bad for Imbalance Data

Recall / True Positive Rate = Correctly classified Actual +ves / Total Actual +ves

Recall ↑ FN ↓

→ when FN is more expensive

= TP / TP + FN → fraction of Spam emails correctly identified as spam

False Positive Rate = FP / FP + TN → basically for non Spam

→ when FP are more expensive

Precision = Correctly classified Actual +ves / Everything classified as +ve = TP / TP + FP

Precision ↑ FP ↓

+ve predictions are how much correct ← fraction of emails classified as Spam that were actually spam

F1 Score = Harmonic Mean of Precision & Recall

$$= 2 * \frac{P * R}{P + R}$$

balances both → more preferred than accuracy when data is imbalanced

F-Beta $= (1+\beta^2)\frac{P \times R}{\beta(P+R)}$

$\beta = 0.5$ FP > FN
$\beta = 2$ FN > FP
$\beta = 1$ → equal

Recall is how much +ves are accurately predicted
Precision is of the predicted +ves how much is correct

(ROC) Receiver Operating Characteristic curve
↓
Visual representation of model performance across all thresholds

TPR v/s FPR for every threshold

(AUC) Area under Curve of ROC
Eg: AUC = 0.5 → 50% probability that model accurately predicts +ves & -ves (random)

ROC & AUC → only when balanced data

PR ROC & AUC → For imbalanced data
Precision - Recall

Threshold with greater area, generally better

Multi Class → for Multiple Classes

More time in evaluating, cleaning & transforming data than building the model

Proper No.s → Temp., weight, etc.
Categorical No.s → Postal Codes like 40004 is not 2 time better 20002

Mode takes values in floating vectors called feature vector [8.6, 132.9]
(then normaliz^n/Binning

.describe() → All Stats like Mean, Median, etc..

Normalization → bringing data to Similar range

!
faster Convergance,
better predictions,
Avoid NaN trap (high values)
Saving computation task

Types
→ Linear Scaling
→ Z-Score "
→ log "

Linear Scaling → $x' = \frac{x - x_{min}}{x_{max} - x_{min}}$

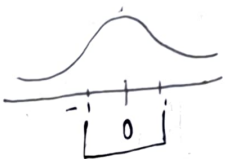
↓ Basic (Normalization)

Range [0, 1]

Z Score Scaling → $x' = \frac{x - \mu}{\sigma}$ → better for Normal Distribution

↓ (Standardization)

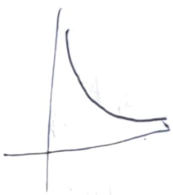
Mean → 0
Std. Dev → 1



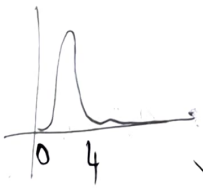
68.27% of data

Log Scaling → $x' = \ln(x)$ → only when distribution is power like

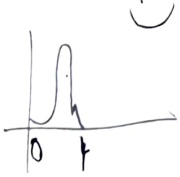
Eg:



Clipping →



→ Most values are in 0 - 4 So the rest are like if greater than 4, you are 4 & if less than 0, you are 0



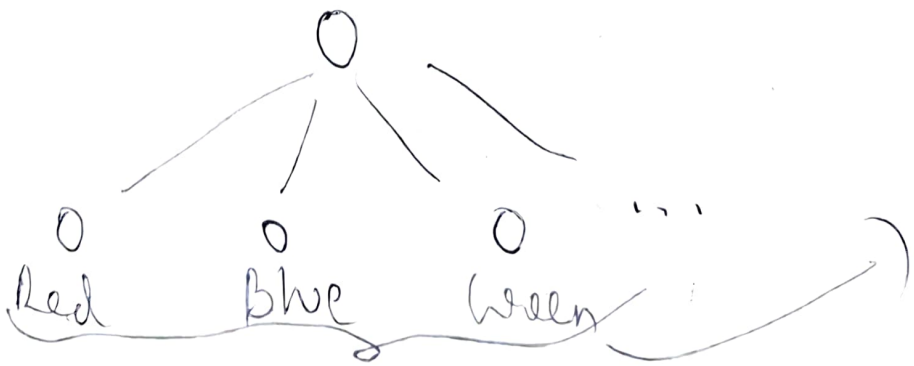
Categorical Data

Eg: Different Species in park
Binning no. etc.

In numerical data, usually 2x feature approximately results in 2 times the label so easy training but for categorical data it is not such so more time in training

Encoding → Converting categorical data into numerical vectors so that model can train

No. of elements → dimension



Indexing → Red 0 [1 0 ...]
Blue 1 [0 1 ...]
Green 2 [0 0 1 ...]
} → One Hot encoding

Multi Hot encoding → [1 1 0 0 ..] (multiple ones)

[0, 0, 1, ...] Sparse Rep^n → 2 One Hot
5 Multi Hot
1
no. of zeros

If outliers like "Avocado" for color ⟶ make them into 1 category like "catch-all"

Down sampling → training on disproportionately low subset of majority class

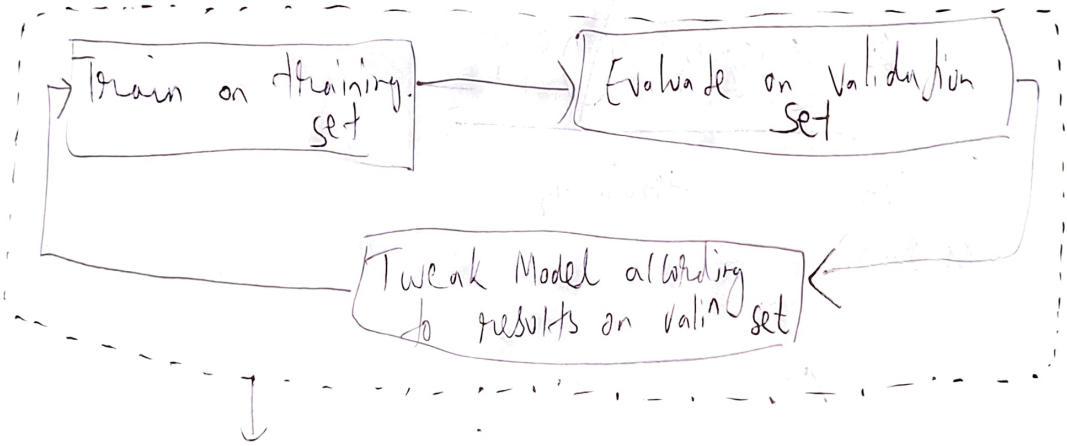
Up weighting → adding example weight to the down sampled class equal to the factor by which you downsampled

Minority : Majority

1 : 200 → 1 : 20 } Down Sampling

Up weighting → Complicated Shit

Data → Train / Validate / Test

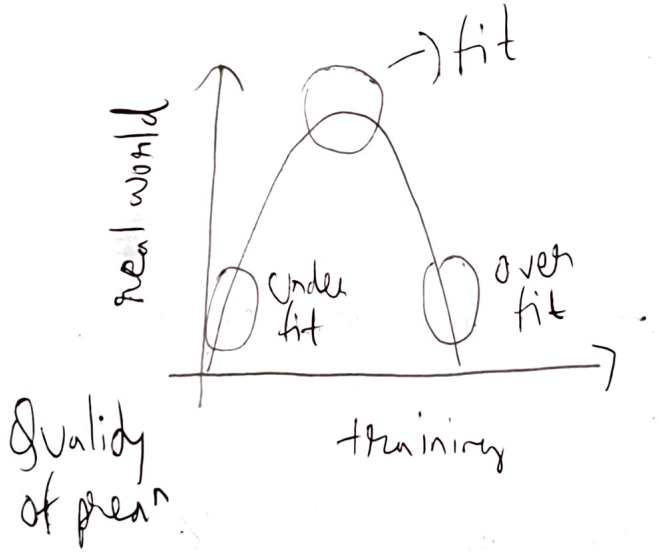


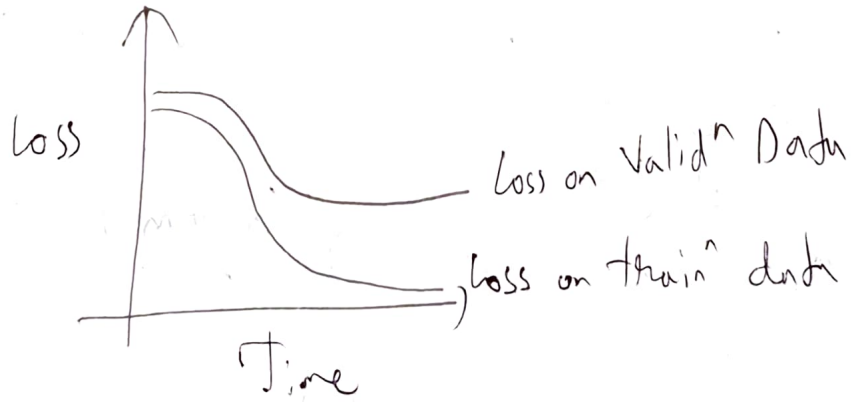
↓ Tells how well it is performing on unseen data for that epoch

Here also giving same data leads to overfitting so Starting a review with new data

Usually the dataset is just a subset of real world data. So usually it tends to fails. So get from more Broader ranges of example & generalize

Overfitting → Memorizing training data





Generally, Simpler outperform Complex models in real world data not in training

Augmentation → Creating new data from given data. Eg: Cat → Mirror it, crop it

For Session Overfitting → Increasing dataset size → Augmentation

Regularization (L1, L2, Dropout) → Nullify the effect of some neurons

Overfitting → Highly Complex Non Linear Curve → Smoothen it by taking off some labels

$$\text{Cost} = \frac{1}{m}\sum_{i=1}^{n} \text{Loss}_i + \sum_{j=1}^{n} \frac{\lambda}{2m} |w_j|^2$$

→ Adding a term

→ L2 (Ridge)

$|w_j|^2$ → weights

λ → Tuner i.e. learning rate like

$$\sum_{j=1}^{n} \frac{\lambda}{m} |w_j|$$

→ L1 (Lasso)

$\frac{\partial \text{cost}}{\partial w}$ → will change as well by → $\frac{\lambda}{m} w_j$

Dropout → D = random(...) < keep_prob

$A = A * D / \text{keep\_prob}$

$\frac{\partial \text{cost}}{\partial w}$ → Also change